Reti Antonio

30232

Mini-StackOverflow
Project Lifecycle Documentation
Version 1.0

Revision History

Date	Version	Description	Author
19.04.2026	1.0	Project Specification & Elaboration – Iteration 1.1	Reti Antonio

1. Project Specification
2. Elaboration – Iteration 1.1
    2.1 Domain Model
    2.2 Architectural Design
        2.2.1 Conceptual Architecture
        2.2.2 Package Design
        2.2.3 Component Diagrams & Deployment Diagrams
3. Elaboration – Iteration 1.2
    3.1 Design Model
        3.1.2 Interaction View
        3.1.2 Class Design
    3.2 Data Model
    3.3 Unit Testing
4. Elaboration – Iteration 2
    4.1 Architectural Design Refinement
    4.2 Design Model Refinement
5. Construction and Transition
    5.1 System Testing
    5.2 Future Improvements
6. Bibliography

1. Project Specification

Purpose and Objectives The **Mini-StackOverflow** system is a specialized Q&A platform designed to facilitate technical knowledge sharing through a structured, client-server architecture. The primary objective is to maintain a high-quality knowledge base by allowing users to contribute content while providing moderators with robust tools to manage community standards.

Problem Domain Overview The system addresses the challenge of information noise and inappropriate content in open forums. It provides a centralized database for persistence and a clear hierarchy of roles to ensure that technical discussions remain relevant and moderated.

High-Level Requirements:

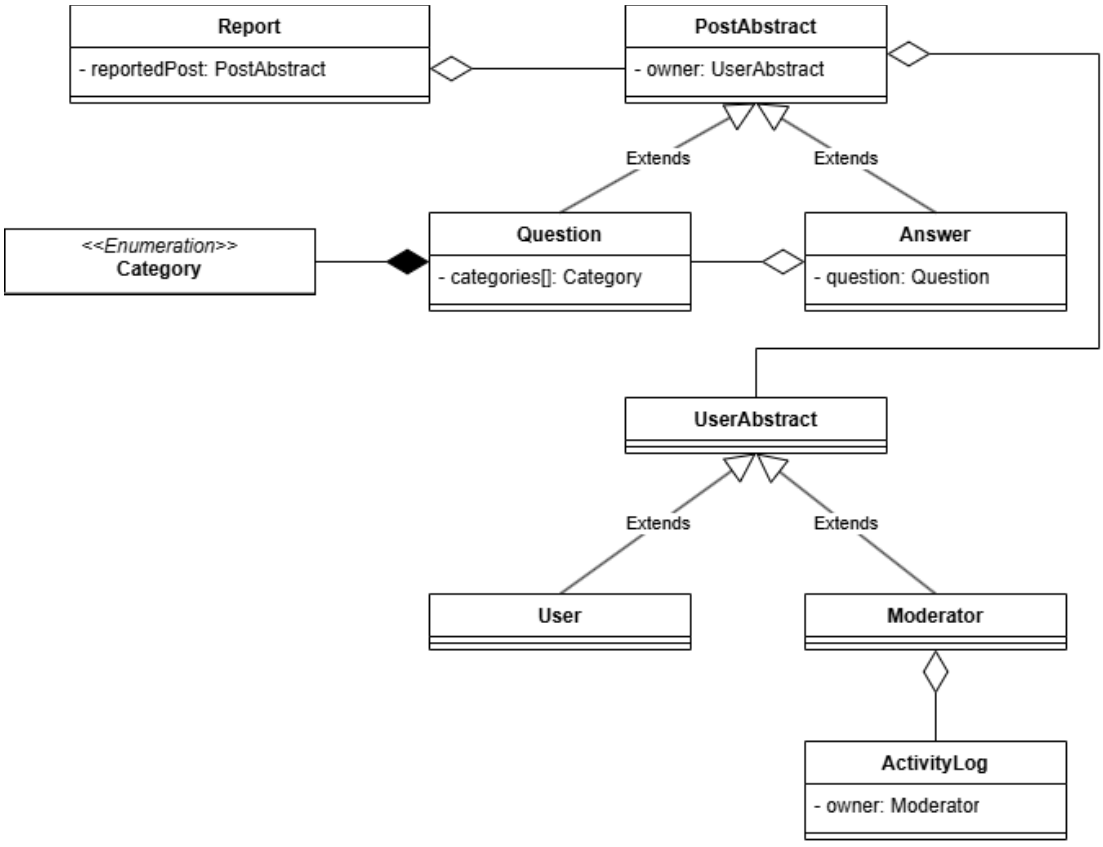
- **User Roles**: Support for Regular Users and privileged Moderators.
- **Post Management**: Creation of questions, posting of answers, and editing capabilities.
- **Moderation Workflow**: Reporting system for flagging inappropriate posts and administrative actions like user suspension.
- **System Intelligence**: Search functionality for existing questions and JSON-formatted activity logging.

2. Elaboration – Iteration 1.1

2.1 Domain Model

The domain model captures the core entities and their relationships within the problem domain.

- **User:** The central entity representing both Regular Users and Moderators.
- **Question:** A technical query created by a user containing tags and categories.
- **Answer:** A response created by a user linked specifically to a single question.
- **Report:** A dedicated model that links a User to a specific Post, carrying a "reason" and a "status" (Pending, Approved, Dismissed) to support the Approve flagged posts requirement.
- **Category (Enum):** A predefined set of technical tags (e.g., JAVA, PYTHON, SQL, C++) that ensures users categorize posts within a controlled taxonomy.
- **ActivityLog:** A persistent record of administrative actions taken by Moderators.



Model Class Diagram

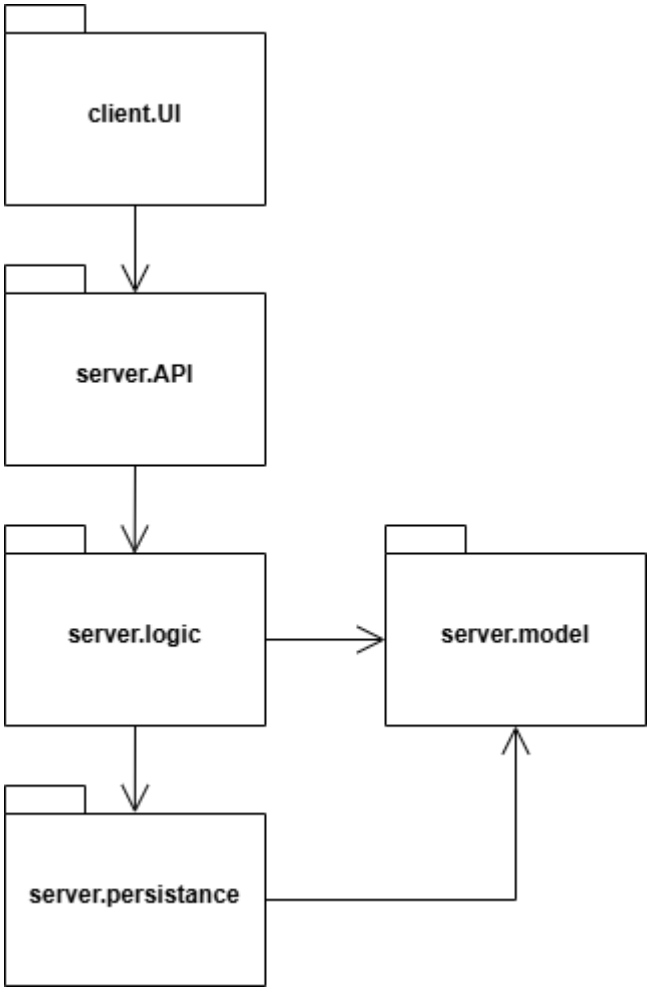
2.2 Architectural Design

2.2.1 Conceptual Architecture

The system utilizes a **Client-Server architectural style** paired with a **Layered Pattern**.

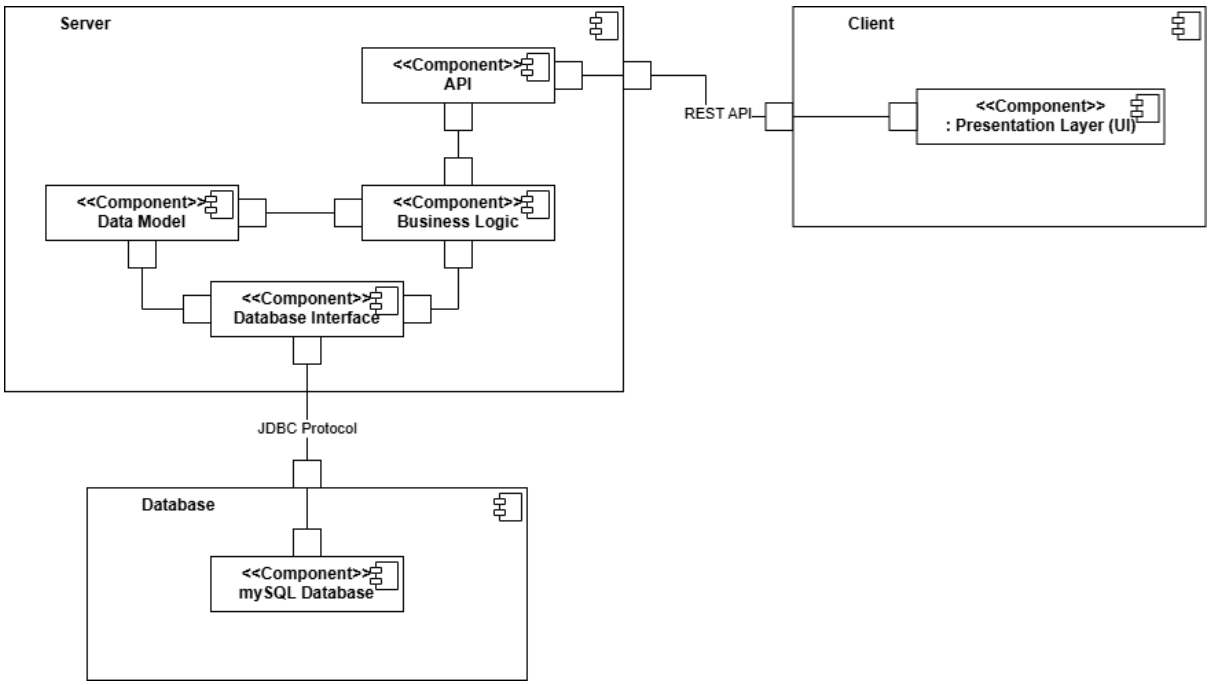
- **Motivation**: This style is chosen to support multiple concurrent users (clients) while maintaining data integrity in a centralized database (server).
- **Application**: The layers (Presentation, Business Logic, and Data Access) ensure that user interactions are decoupled from database operations, enhancing maintainability.

2.2.2 Package Design

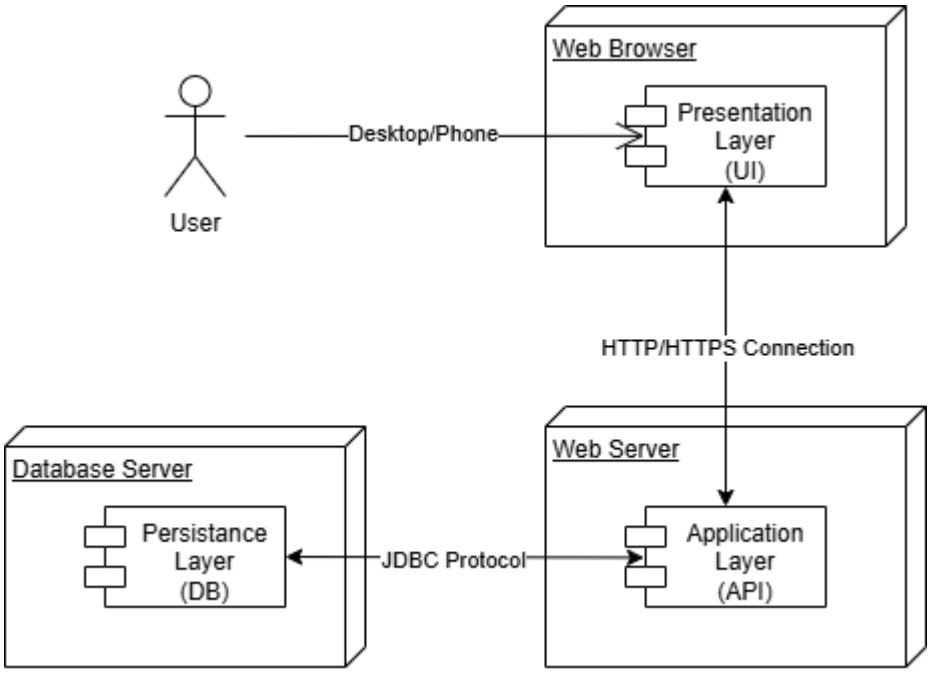


Package Diagram

2.2.3 Component Diagrams & Deployment Diagrams



Component Diagram



Deployment Diagram

3. Elaboration – Iteration 1.2

[Functional Baseline]

3.1 Design Model

3.1.2 Interaction View

[Create interaction diagrams that illustrate how the system behaves at runtime. Include at least a sequence diagram to show the time-ordered exchange of messages between objects or components, and a communication diagram to highlight the relationships and message flow among collaborating elements.]

3.1.2 Class Design

[Create the UML class diagram for the system. Include classes, attributes, methods, and relationships. Apply appropriate object-oriented design patterns where they improve structure or flexibility, and explain the motivation behind the pattern used.]

3.2 Data Model

[Develop the system's data model, including the ER diagram or relational schema, and explain how this structure supports the system's functional and data requirements.]

3.3 Unit Testing

[Present your unit testing strategy, explain the testing techniques you applied, and include the corresponding test case scenarios.]

4. Elaboration – Iteration 2

[Final Architecture Baseline]

4.1 Architectural Design Refinement

[Refine the architectural design using the insights gained from previous iterations. Update (add here, do not change the previous iterations) the conceptual architecture, package structure, component & deployment diagrams, applying relevant design principles. Clearly explain the reasons behind each change.]

4.2 Design Model Refinement

[Add the (new) updated UML class diagram by applying appropriate object-oriented design patterns, Architectural or structural guidelines, GRASP patterns. Explain the reasoning behind your design decisions and include the revised diagrams.]

5. Construction and Transition

5.1 System Testing

[Explain how integration testing was carried out, outlining the methods used and presenting the relevant test case scenarios. Describe how these tests confirm that the system behaves as expected.]

5.2 Future Improvements

[Outline potential future enhancements for the system, describing improvements that could be introduced in later versions.]

6. Bibliography

[List all references used in the creation of this document, including books, articles, websites, and other resources.]